

Criação de Diagramas de Atividades com PlantUML

Como modelar o sistema inteiro sem brigar com o alinhamento as setas.



Estrutura básica de um Diagrama de Atividades em PlantUML

O que é?

- Representa o fluxo de execução de um processo.
- Demonstra a sequência das atividades.
- Permite visualizar o comportamento de uma funcionalidade.
- É utilizado na modelagem de sistemas com UML.

Estrutura básica de um Diagrama de Atividades em PlantUML

@startuml
start
:Entrar no sistema;
:Informar login;
:Verificar dados;
:Mostrar tela principal;
stop
@enduml

Elemento	Função
start	Define o início do fluxo
:atividade;	Representa uma ação
stop	Define o final do fluxo

Estrutura básica de um Diagrama de Atividades em PlantUML



Estrutura básica de um Diagrama de Atividades em PlantUML

Principais pontos

- O PlantUML permite criar diagramas por meio de código.
- A estrutura básica possui início, atividades e fim.
- O diagrama facilita a representação do fluxo de processos.
- Essa estrutura pode ser expandida com decisões, loops e atividades paralelas.

Decisões no PlantUML

Como a maioria das linguagens de programação, o PlantUML possui diversas estruturas de decisão. Elas servem para indicar os fluxos lógicos do programa (ou no caso, o diagrama).

No caso de um Diagrama de Atividades, essas estruturas são utilizadas para mapear fluxos lógicos e algoritmos. Usando palavras-chaves idênticas às linguagens de programação, para tornar o uso padronizado.

Decisões no PlantUML

São elas:

if -> Se

then -> Então

else -> Senão

endif -> Executa de qualquer forma (independentemente da condição)

elseif -> Múltiplas condições

switch/case -> Múltiplas condições

Decisões no PlantUML

If / Else (Condição Simples)

```
@startuml
start
if (O usuário está logado?) then (sim)
    :Mostrar painel principal;
else (não)
    :Redirecionar para login;
endif
stop
@enduml
```

Elseif (Múltiplas Condições)

```
@startuml
start
if (Código == 200?) then (sim)
    :Processar dados;
elseif (Código == 404?) then (sim)
    :Exibir "Página não encontrada";
elseif (Código == 500?) then (sim)
    :Exibir "Erro interno";
else (outro)
    :Exibir "Erro desconhecido";
endif
stop
@enduml
```

Decisões no PlantUML

Switch / Case (Estrutura de Escolha)

@startuml

start

switch (Nível de Acesso)

case (Admin)

:Permitir acesso total;

case (Editor)

:Permitir edição;

case (Leitor)

:Permitir apenas leitura;

endswitch

stop

@enduml

Atividades paralelas

As atividades paralelas (também chamadas de execução concorrente) ocorrem quando duas ou mais tarefas podem ser executadas simultaneamente, pois não dependem do término umas das outras para iniciar.

Atividades paralelas

Esse comportamento é representado graficamente por uma barra de sincronização espessa, dividida em dois conceitos complementares: Fork (bifurcação) e Join (união).

Fork: Representado por uma barra com uma seta de entrada e múltiplas setas de saída. Tem como função dividir um único fluxo de controle em dois ou mais fluxos simultâneos.

Join: Representado por uma barra com diversas setas de entrada e uma seta de saída. Sincroniza e une os fluxos paralelos novamente; o fluxo principal só avança quando todas as atividades anteriores terminam.

Atividades paralelas

Exemplo:

Atividade: Finalizar Pedido.

Entrada no fork: Divide o fluxo para executar paralelamente os caminhos "Processar Pagamento" e "Separar Produtos no Estoque". Após isso, o sistema aguarda ambas as confirmações (Join) para dar continuidade à atividade.

@startuml

:Finalizar Pedido;

fork

:Processar Pagamento;

fork again

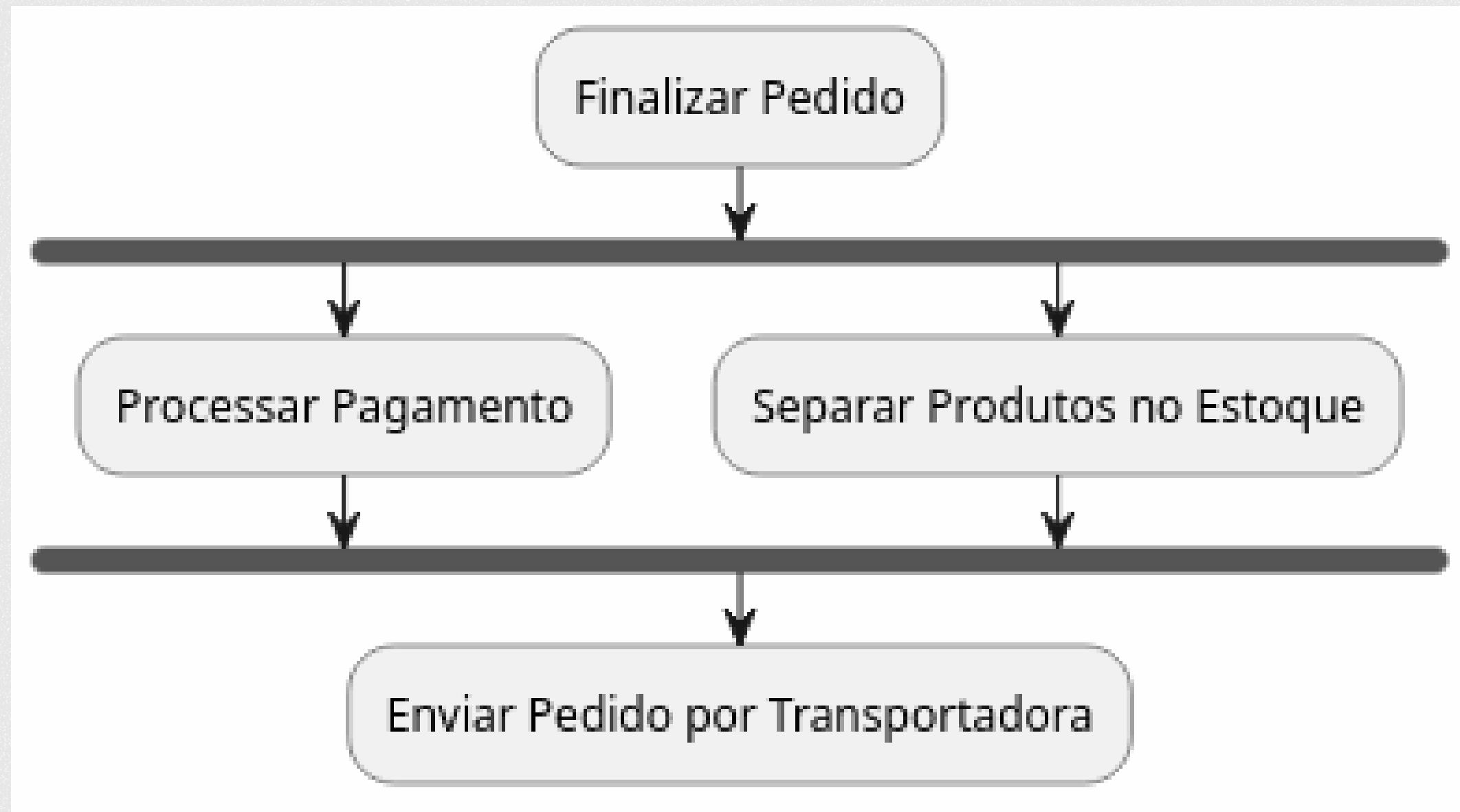
:Separar Produtos no Estoque;

end fork

:Enviar Pedido por Transportadora;

@enduml

Atividades paralelas



Atividades paralelas

Importante:

O *fork* não tem relação com o losango de decisão (*if/else*). Nas atividades paralelas, **TODOS** os caminhos anexados são executados, independentemente se o primeiro levar 10 segundos e o segundo demorar 2 horas.

Atividades paralelas

Tokens:

A UML explica o funcionamento das atividades paralelas usando o conceito de tokens (que podem ser imaginados como fichas ou moedas):

- Uma ação só é iniciada quando recebe um token de entrada.
- Quando a ação é finalizada, ela repassa o token para a próxima etapa.

Atividades paralelas

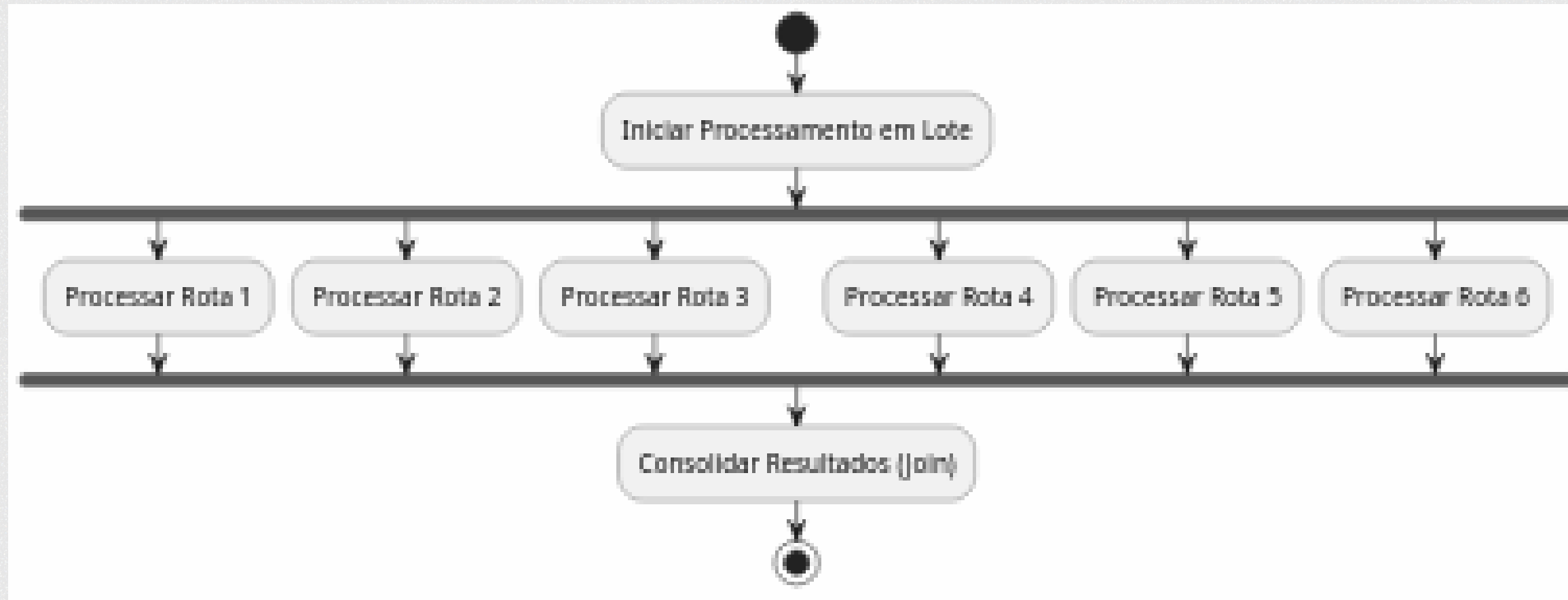
- No Fork, o token é multiplicado pela quantidade de caminhos possíveis. Esse token idêntico percorre todas as ramificações conectadas, fazendo com que as atividades comecem a ser executadas simultaneamente.
- Já o Join funciona como uma barreira de sincronização. Ele só permite que o fluxo continue se receber todos os tokens que foram multiplicados anteriormente. Apenas quando o último token pendente chega, o Join consome todas as entradas, funde-as em um único token e o envia para a próxima atividade do diagrama.

Atividades paralelas

Noções importantes

- **Independência de tempo:** Atividades paralelas não começam e terminam necessariamente no exato mesmo milissegundo; isso significa apenas que a ordem de execução entre elas não importa. A Atividade A pode levar 2 segundos e a Atividade B, 2 horas; o Join garantirá que o sistema aguarde a conclusão de ambas.
- **Sem dependência direta de dados:** Enquanto as tarefas estão sendo executadas em paralelo, o fluxo de uma não pode depender do resultado parcial da outra. Elas operam de forma completamente isolada até se unificarem no Join.

Atividades paralelas



- @startuml
- start
- :Iniciar Processamento em Lote;
-
- fork
- :Processar Rota 1;
- fork again
- :Processar Rota 2;
- fork again
- :Processar Rota 3;
- fork again
- :Processar Rota 4;
- fork again
- :Processar Rota 5;
- fork again
- :Processar Rota 6;
- fork again
- :Processar Rota 7;
- fork again
- :Processar Rota 8;
- fork again
- :Processar Rota 9;
- fork again
- :Processar Rota 10;
- end fork
-
- :Consolidar Resultados (Join);
- stop
- @enduml

Atividades paralelas

```
@startuml
skinparam Archimate {
    BackgroundColor<<ForkJoin>> Black
}
```

title Diagrama de Atividades - Checkout de Passagens Aéreas

```
start
:Receber Requisição de Compra;
:Validar Dados do Formulário;
```

```
note right
    Início do Paralelismo (Fork)
    As 4 rotas rodam simultaneamente
end note
```



```
fork
    :Processar Gateway de Pagamento;
    :Gerar Comprovante de Transação;
fork again
    :Deduzir Pontos do Programa de Milhas;
fork again
    :Reservar Assento na API da Companhia Aérea;
    :Bloquear Poltrona no Voo;
fork again
    :Carregar Template de Notificação (E-mail/SMS);
end fork
```

```
note right
    Barreira de Sincronização (Join)
    Aguarda todas as APIs responderem
end note
```

```
:Emitir Bilhete Eletrônico (e-Ticket);
:Enviar Confirmação ao Cliente;
stop
@enduml
```



Laços de Repetição (Loops)

No Diagrama de Atividades da UML, os laços de repetição representam trechos do fluxo que devem ser executados repetidamente até que uma condição específica seja atendida.

Existem duas maneiras oficiais para representar loops na UML: usando Nós de Decisão/União convencionais (a abordagem clássica) ou usando um Nó de Estrutura de Loop (Loop Structured Activity Node — a abordagem moderna que encapsula o bloco).

Laços de Repetição (Loops)

1. A Abordagem Clássica: Nós de Decisão e União (Merge)

Esta é a forma mais comum e visual, baseada em fluxogramas e setas de retorno. Ela utiliza dois elementos em formato de losango:

- **União (Merge):** Posicionado no início do loop, serve para receber tanto o fluxo principal do diagrama quanto o fluxo que está retornando da repetição.
- **Decisão (Decision):** Posicionado após a atividade para avaliar a Condição de Guarda (texto escrito entre colchetes [...]). Se a condição for verdadeira, o fluxo retorna para o Merge; se for falsa, o loop é encerrado e o fluxo principal segue adiante.

Laços de Repetição (Loops)

@startuml

title Loop Clássico - Processamento de Itens do Carrinho

start

:Iniciar Fechamento de Pedido;

repeat

:Selecionar Próximo Item do Carrinho;

:Calcular Frete do Item;

:Reservar Item no Estoque;

backward:Ir para o próximo item;

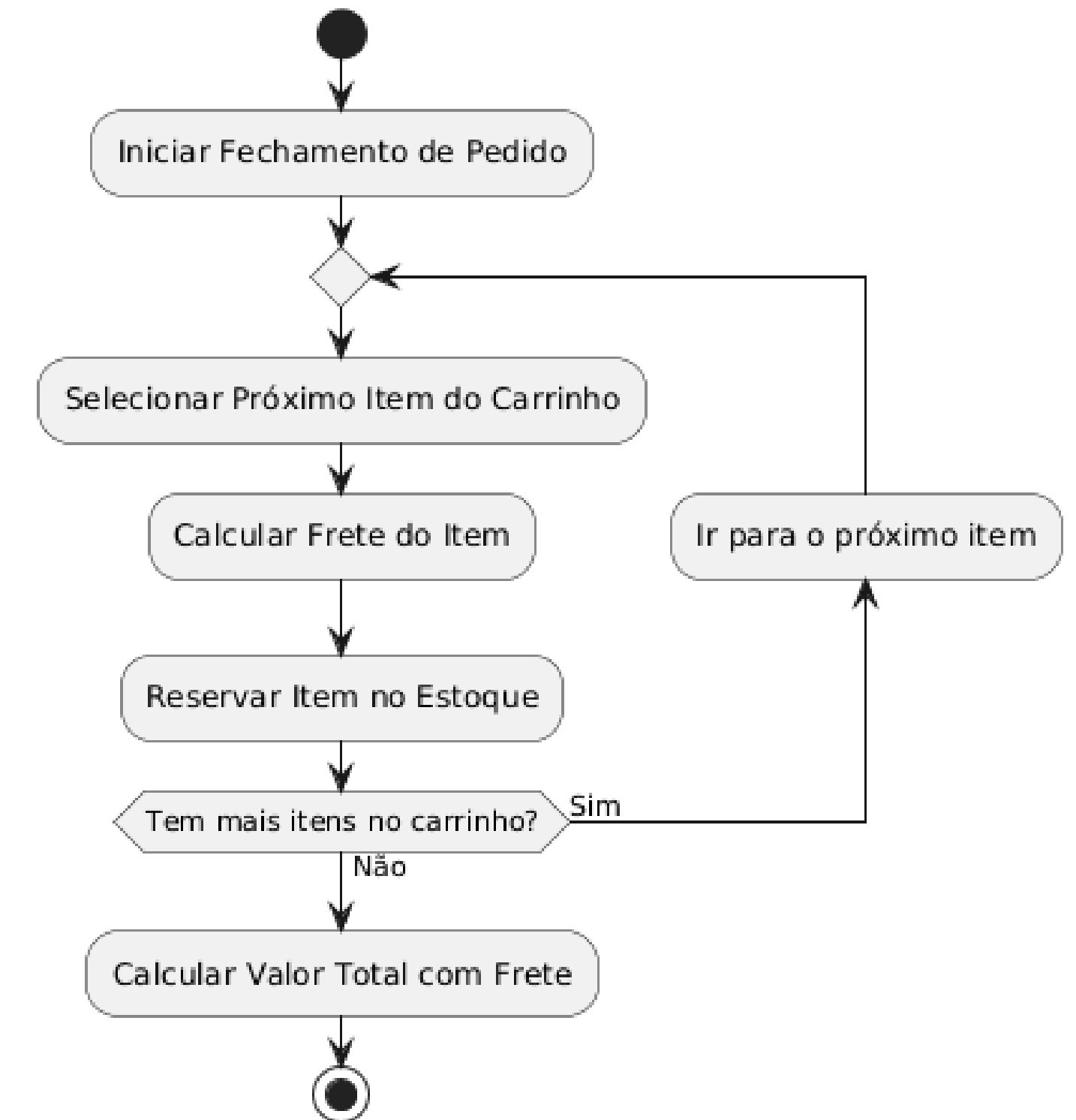
repeat while (Tem mais itens no carrinho?) is (Sim) not (Não)

:Calcular Valor Total com Frete;

stop

@enduml

Loop Clássico - Processamento de Itens do Carrinho



Laços de Repetição (Loops)

Abordagem Moderna: Nó de Atividade Estruturada (Loop Node)

Para diagramas complexos, a UML introduziu o Nó de Atividade Estruturada de Repetição (Loop Structured Activity Node). Em vez de utilizar setas de retorno cruzando o diagrama, desenha-se um retângulo de borda tracejada ao redor das atividades que se repetem. Esse retângulo possui compartimentos internos opcionais:

Setup (Configuração): Ações de inicialização (ex.: $i = 0$).

Test (Teste): A condição de avaliação ou parada (ex.: $i < 10$).

Body (Corpo): As ações que são executadas repetidamente.

Laços de Repetição (Loops)

@startuml

title Nó de Atividade Estruturada -
Loop Node (Abordagem Moderna)

start

:Carregar Arquivo XML com 50 Notas Fiscais;

group loop [Para cada Nota Fiscal encontrada]

:Ler Dados do Emitente;

:Calcular Imposto Retido (ISS/ICMS);

:Enviar Dados para a SEFAZ;

end group

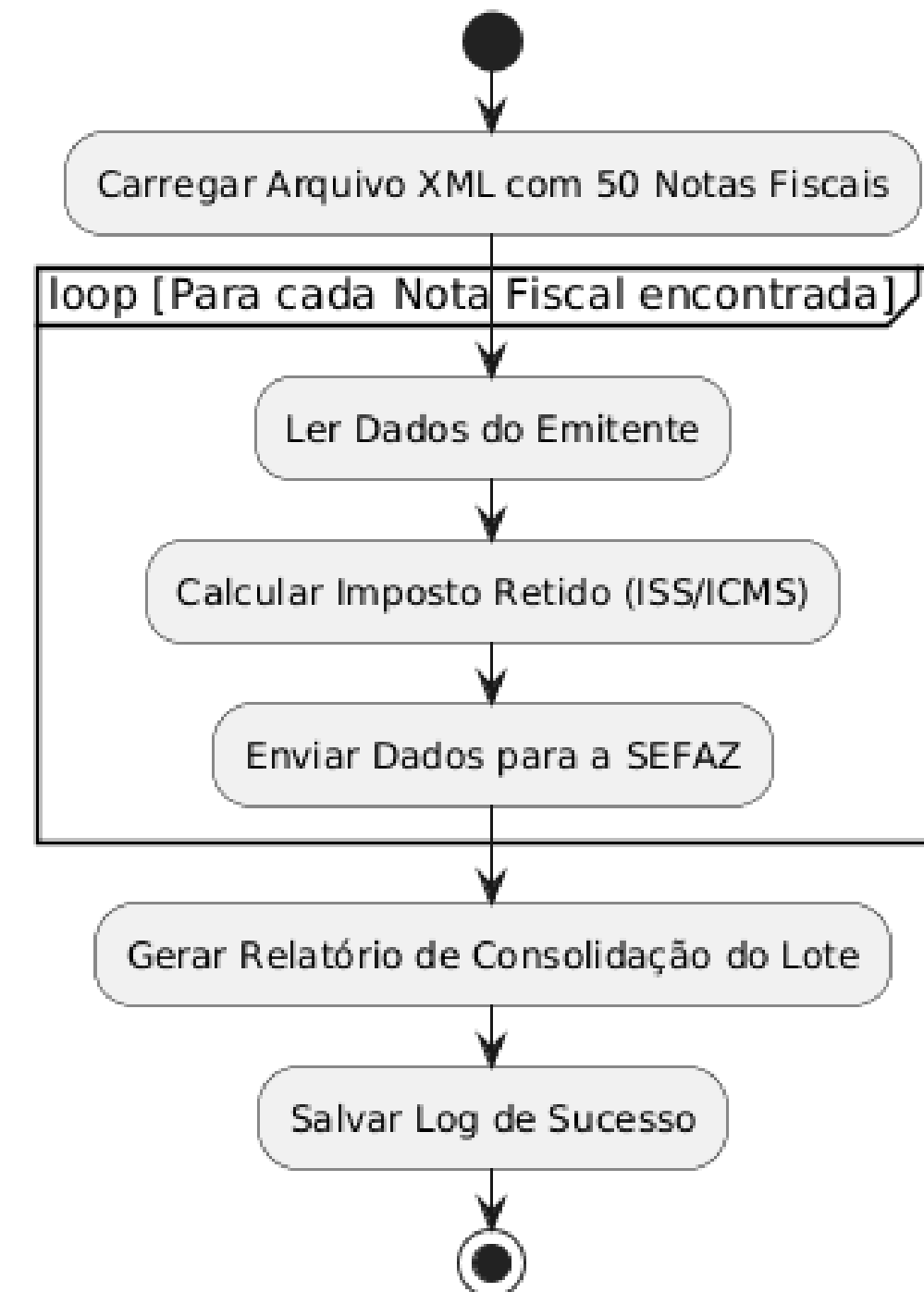
:Gerar Relatório de Consolidação do Lote;

:Salvar Log de Sucesso;

stop

@enduml

Nó de Atividade Estruturada - Loop Node (Abordagem Moderna)



Raias de responsabilidade

É um fluxograma usado para organizar ou planejar o fluxo de controle de um projeto, que ajuda a separar as tarefas.

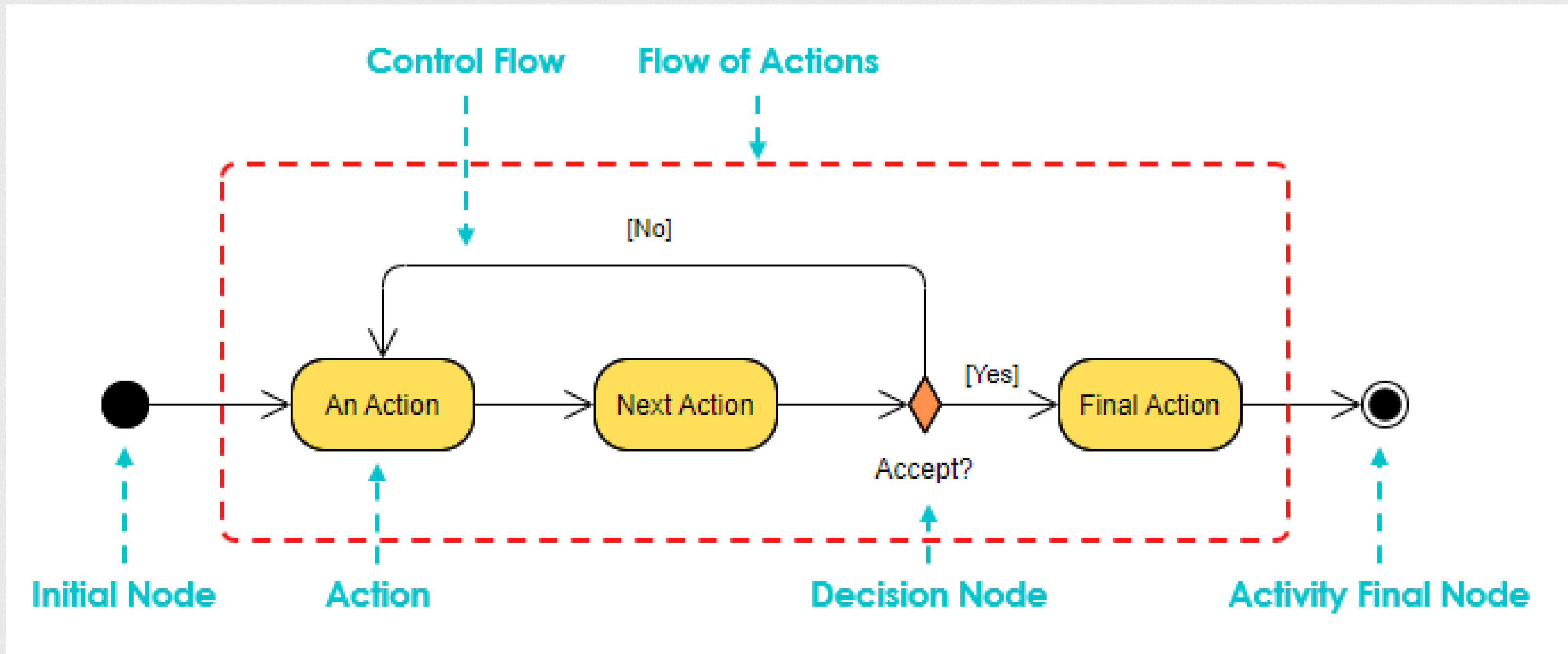
Identificar os Atores e responsabilidades
- Cada entidade será representada por uma raia (faixa horizontal ou vertical). Isso permite visualizar imediatamente a divisão de trabalho e as fronteiras de responsabilidade.

Mapeamento do fluxo de controle - Dentro de cada raia é inserido as ações que cada ator executa

Raias de responsabilidade

As raias podem ser organizadas de forma hierárquica ou bidimensional, cada um agrupa as ações, as setas podem atravessar a raia demonstrando interação e transferência de responsabilidade entre os atores.

Raias de responsabilidade





Notas explicativas

O que são e para que servem?

As notas explicativas (ou comentários) são recursos utilizados no PlantUML para documentar regras de negócio, esclarecer processos complexos ou deixar lembretes no diagrama. Elas não alteram a lógica ou o fluxo principal das atividades, mas são essenciais para quem está lendo e interpretando a modelagem.

Notas explicativas

Sintaxe e tipos de notas no PlantUML:

1. Notas Simples (Linha Única):

Ficam anexadas à última atividade definida. Podem ser posicionadas à direita ou à esquerda.

Comandos:

note right: Seu texto

note left: Seu texto

2. Notas de Múltiplas Linhas (Blocos):

Usadas quando a explicação é mais longa e requer quebras de linha para não poluir visualmente o diagrama.

Comando:

note right

Linha 1 da expliação

Linha 2 da explicação

end note

Notas explicativas

3. Notas Flutuantes (Floating Notes):

São notas que não ficam amarradas a uma atividade específica.

Servem para dar um aviso geral sobre uma área do diagrama.

Comando: floating note:

Aviso importante sobre o fluxo

Código de exemplo prático:

@startuml
start

:Receber Pedido;
note right: Chega via integração com o site

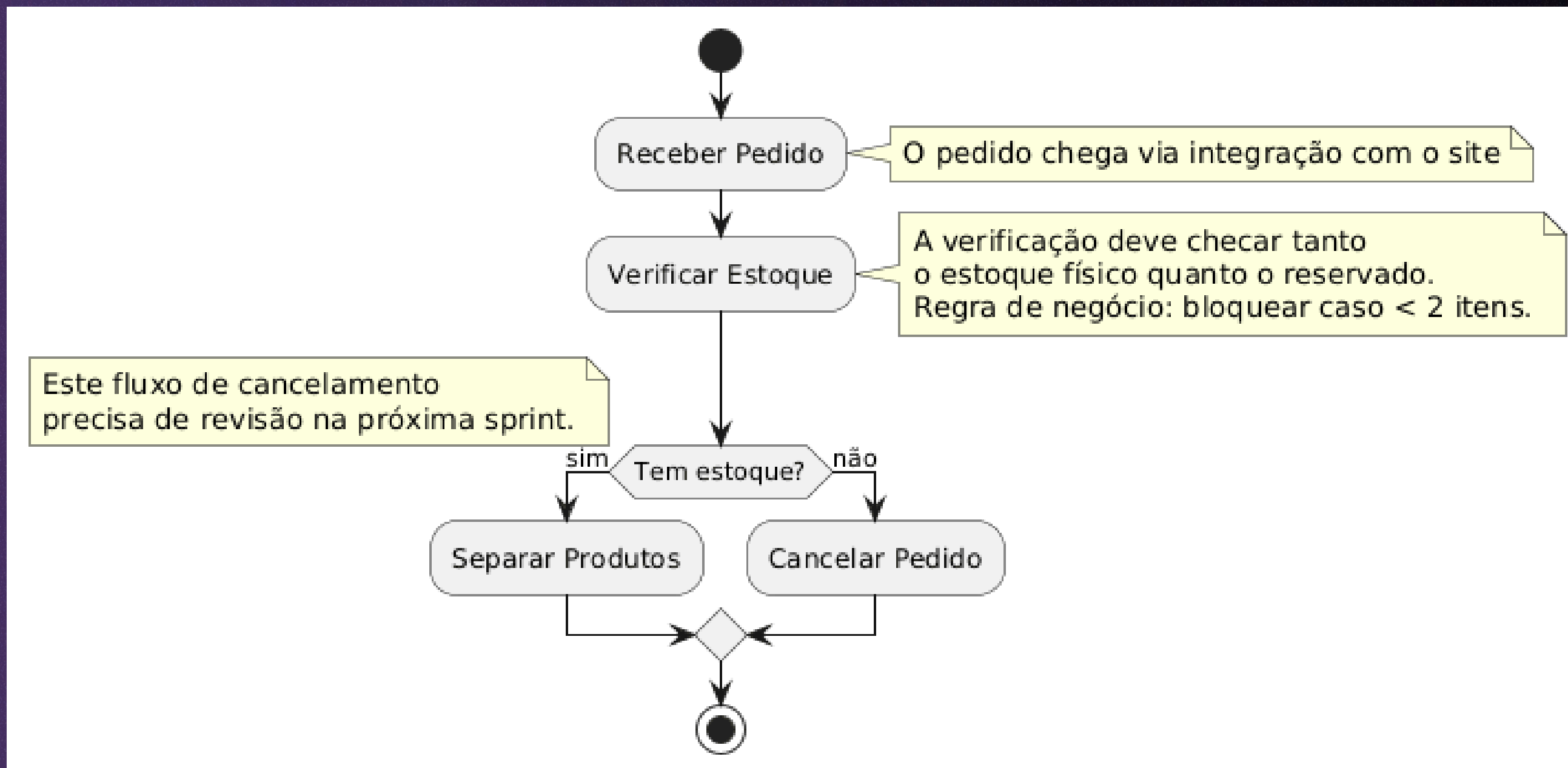
```
:Verificar Estoque;  
note right  
    A verificação deve checar tanto  
    o estoque físico quanto o reservado.  
    Regra: bloquear caso < 2 itens.  
end note
```

```
if (Tem estoque?) then (sim)
    :Separar Produtos;
else (não)
    :Cancelar Pedido;
endif
```

floating note: Este fluxo de cancelamento precisa de revisão na próxima sprint.

stop
@enduml

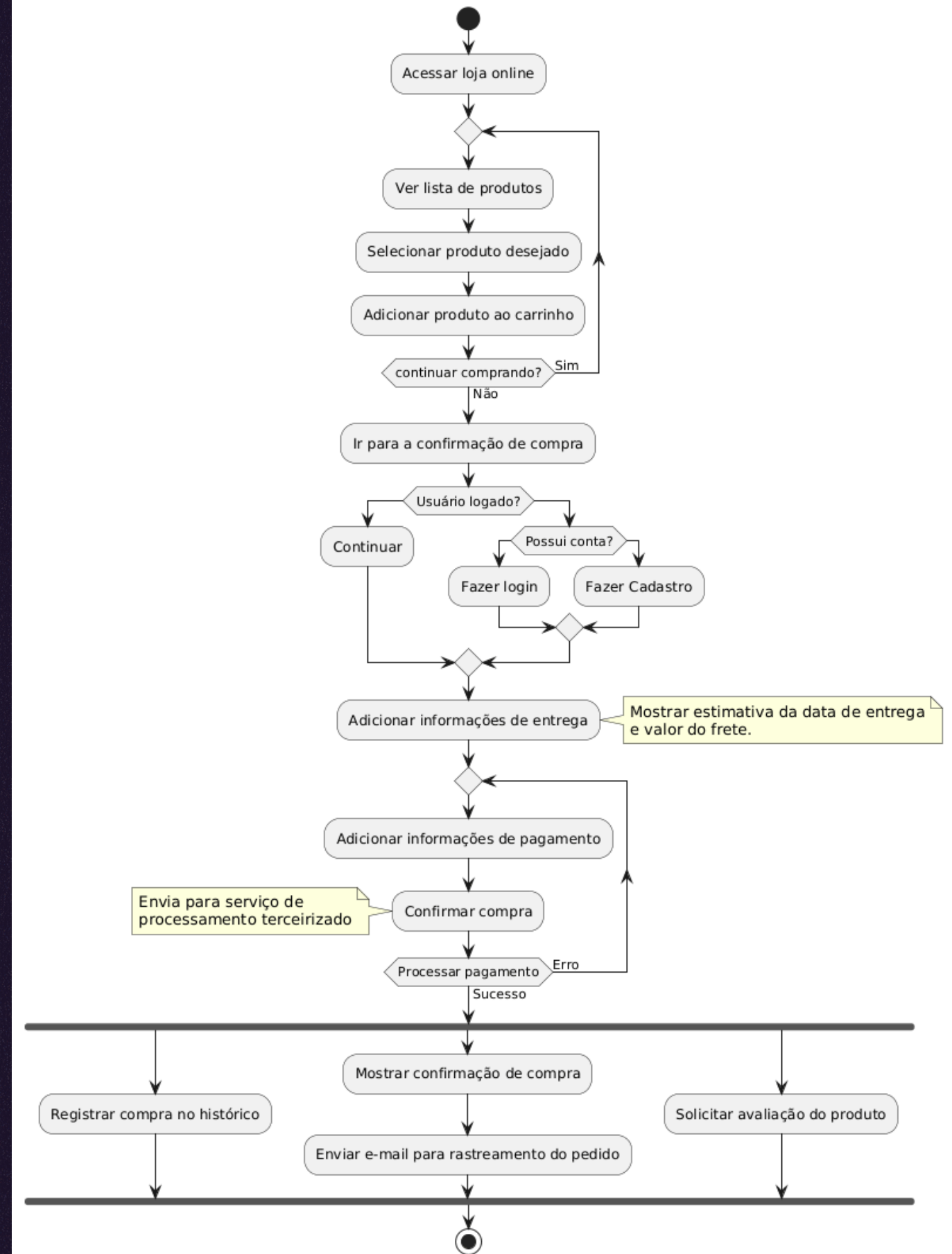
Notas explicativas

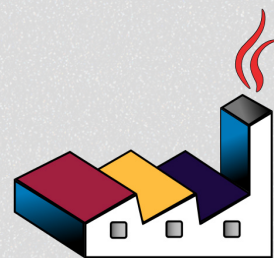


Exemplos de implementação do diagrama de atividades em PlantUML

Exemplos de implementação do diagrama de atividades em PlantUML

Exemplo de Diagrama de atividades: Loja Online





Como diria a minha ex:
TERMINAMOS

